

Installing and Running

From `module_5`, create a virtual environment and install the pinned dependencies plus the project itself.

With pip:

1. `python3 -m venv venv`
2. `source venv/bin/activate`
3. `pip install -r requirements.txt`
4. `pip install -e .`

With uv:

1. `uv venv venv`
2. `source venv/bin/activate`
3. `uv pip sync requirements.txt`
4. `uv pip install -e .`

`requirements.txt` pins every package, including indirect dependencies. It is generated from `requirements.in` with the command below, so `uv pip sync` reproduces the same environment exactly on macOS and Linux:

- `uv pip compile requirements.in --universal --python-version 3.12 -o requirements.txt`

After copying `.env.example` to `.env` and filling in the `DB_*` credentials, start the app with:

- `cd src`
- `python run.py`

Then, open the `/analysis` endpoint.

Why packaging matters

`setup.py` makes the project installable, so its packages (`app`, `database`, `scraping`, `llm_hosting`) import the same way in all places, whether it's local runs, the test suite, CI or background subprocesses. Without `setup.py`, imports depend on the current directory or on editing `sys.path`. `pip install -e .` links the environment to the source tree, so code changes take effect without reinstalling, and it avoids path problems caused by differences between machines. `setup.py` declares the project's needs as

compatible version ranges (`install_requires`), while `requirements.txt` records the one exact, tested environment. Having both `setup.py` and `requirements.txt` separates what the package needs from how this deployment is reproduced. Tools like `uv` can also read `setup.py`.

Dependency Graph

The entry point of the graph is `run.py` at the bottom, which imports the Flask `app` package, whose `app.routes` module serves the analysis page and its button endpoints while `app.pull_control` runs the scraper as a background process. Both depend on the `database` package, where `database.db_helpers` reads the `DB_*` credentials, opens connections, and holds the shared query limit and table identifier imported by `load_data`, `query_data`, `models`, and `orm_queries`. `query_data` and `load_data` interact with PostgreSQL using `psycopg`, the large cluster on the right, using its connection and cursor modules plus `psycopg.sql` to compose every statement with safely quoted identifiers and bound parameters. `models` and `orm_queries` use SQLAlchemy, the large cluster on the left, whose `engine`, `orm`, and `sql` modules map the `applicants` table and build the ORM version of the analysis that the web page displays, and Flask renders that page. On the scraping side, `scraping.scrape` drives Chrome with Selenium and parses each results page with BeautifulSoup (`bs4`). The scraped rows are turned into structured results by `scraping.clean` and saved by `scraping.storage`, and `load_data` reuses storage to validate input files. The small `paths` module is shared by nearly every package for file locations, and imports only flow from the web layer down to the data layer, never back up, which keeps the database code reusable from the command-line scripts.

Least-privilege database configuration

The application connects as `gradcafe_app`, a dedicated role created by `least_privilege.sql`. It is not a superuser and does not own any tables. Credentials are read only from the `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` and `DB_PASSWORD` environment variables, loaded from a gitignored `.env` file. `.env.example` is a sample `.env` file with variable names and placeholder values.

Granted Permission	Why it is needed
LOGIN	The application must be able to connect as this role
CONNECT on the database	Lets the role open a connection to the database
USAGE on schema `public`	Lets the role find the applicants table in the schema
SELECT on `applicants`	Every analysis question, the school search for query A3, and upsert's conflict check need to read rows
INSERT on `applicants`	“Pull Data” and “Update Analysis” buttons, as well as `load_data`, add newly scraped results
UPDATE on `applicants`	Upsert refreshes existing results, and the cleanup step nulls invalid scores and normalizes nationality

Withheld Permission	Why it is withheld
Superuser, CREATEDB , CREATEROLE , REPLICATION , BYPASSRLS	The app never manages databases, roles or replication, so a stolen credential can't be used to take over the server
DELETE , TRUNCATE	The app never removes rows, so an injection or a bug can't wipe data
DROP , ALTER (ownership)	The table is owned by the admin account, so the app can't change or destroy the schema
CREATE on schema `public`	The app never creates tables, so it cannot add objects
REFERENCES , TRIGGER	Not used by the app

The app itself cannot be read-only. PostgreSQL only checks the connecting role, not which buttons are pressed, and “Pull Data” and “Update Analysis” write through the same connection as the page. These limits were verified by connecting as `gradcafe\_app`. **SELECT**, **INSERT** and **UPDATE** succeed. **DELETE** and **TRUNCATE**

fail with "permission denied for table applicants". **DROP** and **ALTER** fail with "must be owner of table applicants". **CREATE TABLE** fails with "permission denied for schema `public`".

SQL used for least-privileges role (from `least\_privilege.sql`)

```
CREATE ROLE gradcafe_app LOGIN;
ALTER ROLE gradcafe_app LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE
NOREPLICATION NOBYPASSRLS;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT CONNECT ON DATABASE cam_db TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
REVOKE ALL ON TABLE applicants FROM PUBLIC;
REVOKE ALL ON TABLE applicants FROM gradcafe_app;
GRANT SELECT, INSERT, UPDATE ON TABLE applicants TO gradcafe_app;
```

Dependency Vulnerability Scan

Snyk scanned 76 pinned dependencies from `requirements.txt` against its open-source vulnerability database. The `requirements.txt` was compiled with `uv pip compile --universal`. The "--universal" flag adds Windows-only platform markers for packages like `cffi` and `pycparser`. Snyk's resolver fails on these conditional entries when run on macOS, so those two lines needed to be stripped before scanning. As such, they are not present in the macOS runtime environment and carry no risk there. Snyk found one high severity issue across 2 vulnerable paths:

Severity	Package	Version	Finding
HIGH	`diskcache`	5.6.3	Deserialization of Untrusted Data (SNYK-PYTHON-DISKCACHE-15268422)

`diskcache` is a transitive dependency introduced by `llama-cpp-python`. It uses the `pickle` module to serialize cached objects to disk, which can execute arbitrary code when deserializing maliciously crafted data. No patched version is available. In this project, the cache only stores the local LLM model responses (no inputs), which reduces practical risk. Eliminating the vulnerability entirely would require replacing `llama-cpp-python` with an LLM library that does not depend on `diskcache`.

Static Code Analysis (Extra Credit)

Snyk Code performed static analysis on 16 findings across `module_5/src/`:

- **SQL Injection (HIGH):** This is a false positive. Snyk flags `app/routes.py` line 109 where `request.args.get("school")` flows into a database call. The input passes through `normalize_school()`, which enforces a 100-character limit and rejects non-printable characters, before reaching `fetch_accepted_since()`. That function builds the query with psycopg's `sql.SQL` composition and passes the school name as a bound parameter after escaping LIKE wildcards with `escape_like()`. No string interpolation into SQL occurs. Snyk SAST cannot trace sanitization through function calls, so it flags this false positive.
- **SSRF (MEDIUM):** This is also a false positive. Snyk flags `scraping/scrape.py` line 386 where a command-line URL is passed to `urllib.request.urlopen` to fetch `robots.txt`. Because the scraper is an internal operator tool and not a web endpoint, the URL is supplied by a trusted operator, and only its `netloc` component is used to construct the `robots.txt` path. Without context that the scraper is only accessible to CLI operators, this gets flagged as a false positive.
- **Path Traversal (LOW, 14 findings):** All 14 findings are in internal scripts `llm_helper.py`, `scrape.py`, and `load_data.py` which accept file paths from CLI arguments. These scripts are run by trusted operators, not exposed as web endpoints. The paths are always operator-supplied and never derived from user input.

GitHub Actions CI

The workflow in `.github/workflows/module_5_ci.yml` runs on every push or pull request that touches `module_5/`. It has four independent jobs:

- Pylint installs the pinned dependencies and the project package, then runs the following command from `module_5/src`, which fails if the score is below 10:
 - `pylint app database scraping llm_hosting paths.py run.py --fail-under=10`
- Dependency graph installs Graphviz and the project dependencies, then runs the following command, failing if a graph cannot be produced:

- ``pydeps src --max-bacon 3 --max-module-depth 2 --no-show from module_5/``
- Snyk dependency scan installs the Snyk CLI and strips the two Windows-only lines from ``requirements.txt`` before scanning because Snyk's resolver cannot handle platform-conditional packages when run on Linux. The scan result mirrors the local scan report, with one HIGH severity issue in ``diskcache``. The ``SNYK_TOKEN`` is stored as a GitHub Actions repository secret.
- Pytest spins up a PostgreSQL 16 service container, loads ``module_5/schema.sql`` to create the ``applicants`` table, then runs the full test suite with coverage. Database credentials are passed as ``DB_*` environment variables pointing at the service container. ``DB_NAME`` is forced to ``cam_db_test`` by the test configuration, so the job can never touch a production database.